

DATA STRUCTURES & ALGORITHMS

A Mathematical Guide to Algorithm Design

Understanding the Mathematical Foundations Without Code

Edition: 2026

TABLE OF CONTENTS

1. Introduction: Why Mathematics Matters in Algorithms
2. Complexity Analysis: Big O Notation & Growth Functions
3. Arrays & Lists: Linear Data Structures
4. Linked Lists: Sequential Access Structures
5. Stacks: Last-In-First-Out Principle
6. Queues: First-In-First-Out Principle
7. Trees: Hierarchical Structures
8. Binary Search Trees: Ordered Trees
9. Balanced Trees: AVL & Red-Black Trees
10. Heaps: Priority-Based Structures
11. Graphs: Networks & Relationships
12. Hash Tables: Constant-Time Lookup
13. Sorting Algorithms: Arranging Data Efficiently
14. Searching Algorithms: Finding Needles in Haystacks
15. Recursion & Backtracking: Divide and Conquer
16. Dynamic Programming: Optimal Substructure
17. Greedy Algorithms: Making Optimal Choices
18. Network Flows & Matching: Advanced Topics

Chapter 1: Introduction - Why Mathematics Matters

1.1 What is an Algorithm?

An algorithm is a finite sequence of well-defined steps that transforms an input into an output. Mathematics provides the language to describe, analyze, and compare algorithms precisely.

1.2 The Three Questions

When designing algorithms, we must ask:

Question 1: Correctness - Does the algorithm produce the right answer?

Mathematics answer: Proof by induction or contradiction

Question 2: Efficiency (Time) - How fast does it run?

Mathematics answer: Time complexity analysis using Big O notation

Question 3: Efficiency (Space) - How much memory does it need?

Mathematics answer: Space complexity analysis

Chapter 2: Complexity Analysis - Big O Notation

2.1 Introduction to Growth Functions

When we analyze an algorithm, we want to know how its performance changes as the input size grows. We express this relationship using mathematical notation.

2.2 Mathematical Definition of Big O

We say $f(n)$ is $O(g(n))$ if there exist constants c and n_0 such that: $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$

Meaning: $f(n)$ grows no faster than $g(n)$ (multiplied by some constant c) for sufficiently large n .

2.3 Common Complexity Classes

Notation	Name	Example	Growth Pattern
$O(1)$	Constant	Accessing array element by index	Flat line
$O(\log n)$	Logarithmic	Binary search	Slow growth
$O(n)$	Linear	Simple loop through n items	Proportional
$O(n \log n)$	Linearithmic	Efficient sorting	Moderate
$O(n^2)$	Quadratic	Nested loops	Fast growth
$O(n^3)$	Cubic	Triple nested loops	Faster growth
$O(2^n)$	Exponential	Subset generation	Very fast growth
$O(n!)$	Factorial	All permutations	Extremely fast growth

2.4 Why Logarithms Matter

The logarithm base 2, written as $\log_2(n)$, answers: 'How many times must I divide n by 2 to get 1?' This is fundamental to algorithms like binary search. For example: $\log_2(1,000,000) \approx 20$, meaning we can search a million items in just 20 comparisons!

2.5 Other Complexity Notations

Theta (Θ): Tight bound - $f(n) = \Theta(g(n))$ means $f(n)$ grows exactly like $g(n)$

Omega (Ω): Lower bound - $f(n) = \Omega(g(n))$ means $f(n)$ grows at least like $g(n)$

Most commonly, we use Big O for worst-case analysis.

Chapter 3: Arrays & Lists - Linear Data Structures

3.1 Mathematical Model of Arrays

An array A of size n can be represented as: $A = [a_0, a_1, a_2, \dots, a_{n-1}]$
Each element a_i occupies memory at address: $\text{base_address} + i \times \text{element_size}$

Key Property: Direct indexing allows $O(1)$ access to any element because we can calculate its memory address directly.

3.2 Array Operations & Complexity

Operation	Time Complexity	Reason
Access element at index i	$O(1)$	Direct address calculation
Search unsorted array	$O(n)$	May need to check all elements
Insert at beginning	$O(n)$	Must shift all elements right
Insert at end (space available)	$O(1)$	Just add to end
Delete from middle	$O(n)$	Must shift remaining elements
Find minimum/maximum	$O(n)$	Must examine all elements

3.3 Example: Finding Maximum Element

Problem: Given an unsorted array A of n integers, find the maximum value.

Analysis: We must examine every element at least once (lower bound: $\Omega(n)$).
By checking all n elements, we can find the maximum in exactly $n-1$ comparisons.
Therefore, the time complexity is $\Theta(n)$ - both upper and lower bounds are linear.

Chapter 4: Linked Lists - Sequential Access Structures

4.1 Mathematical Model of Linked Lists

A linked list is a recursive data structure defined as:
LinkedList = null OR (value, pointer_to_next_LinkedList)

Each node N contains: $N = (\text{data}, \text{link})$ where 'link' points to the next node.

4.2 Key Difference from Arrays

Unlike arrays, linked lists do **not** support random access. To reach element at position k , we must traverse: $\text{node}_1 \rightarrow \text{node}_2 \rightarrow \dots \rightarrow \text{node}_k$, which takes $O(k)$ time.

4.3 Linked List Operations

Operation	Time Complexity	Notes
Access element at index i	$O(i)$	Must traverse from head
Search for value	$O(n)$	Linear search required
Insert at head	$O(1)$	Just update one pointer
Insert at position i	$O(i)$	Must traverse to position
Delete from head	$O(1)$	Update head pointer
Delete from position i	$O(i)$	Must traverse to position

4.4 When to Use Linked Lists

Advantages: $O(1)$ insertion/deletion at known position, dynamic size, no wasted space

Disadvantages: $O(n)$ access time, extra memory for pointers

Use linked lists when you need frequent insertions/deletions; use arrays when you need random access.

Chapter 5: Stacks - Last-In-First-Out Principle

5.1 Mathematical Definition

A stack S is an abstract data type with the property: LIFO (Last-In-First-Out)

Formally, for any sequence of operations $\text{push}(x)$, $\text{pop}()$, the stack ensures that the last item pushed is the first item popped.

5.2 Stack Operations

push(x): Add element x to the top of the stack - $O(1)$

pop(): Remove and return the top element - $O(1)$

peek(): View top element without removing - $O(1)$

isEmpty(): Check if stack is empty - $O(1)$

size(): Return number of elements - $O(1)$

5.3 Real-World Applications

Function Call Stack: Each function call is pushed; when it returns, it's popped.

Undo/Redo: Each action is pushed onto a stack; undo pops from the stack.

Expression Evaluation: Convert infix notation $(a+b)$ to postfix $(ab+)$ using stacks.

Parenthesis Matching: Check if brackets are balanced by pushing/popping.

5.4 Example: Balanced Parentheses Problem

Problem: Given a string of parentheses, determine if they are balanced.

Mathematical Principle: For a balanced sequence, at no point should the number of closing brackets exceed opening brackets. More formally: for a valid sequence S , if we replace each '(' with +1 and each ')' with -1, the cumulative sum never goes negative and ends at 0.

Algorithm: Use a stack. For each '(', push. For each ')', pop if stack non-empty. Valid if all ')' have matching '(' and stack is empty at end.

Complexity: $O(n)$ time, $O(n)$ space in worst case.

Chapter 6: Queues - First-In-First-Out Principle

6.1 Mathematical Definition

A queue Q is an abstract data type with the property: FIFO (First-In-First-Out)
Elements are added at the 'rear' and removed from the 'front'.
Formally: `enqueue(x)` adds x ; `dequeue()` removes the oldest element.

6.2 Queue Operations

`enqueue(x)`: Add element x to the rear - $O(1)$
`dequeue()`: Remove and return the front element - $O(1)$
`peek()`: View front element without removing - $O(1)$
`isEmpty()`: Check if queue is empty - $O(1)$

Circular Queue Optimization: Use modulo arithmetic to reuse space: $\text{rear} = (\text{rear} + 1) \bmod \text{capacity}$, avoiding $O(n)$ space wastage.

6.3 Real-World Applications

Process Scheduling: Operating systems use queues to manage process execution.
Breadth-First Search (BFS): Explore graph level by level using a queue.
Print Queue: Documents are printed in the order they were submitted.
Customer Service: Service customers in the order they arrive.

Chapter 7: Trees - Hierarchical Structures

7.1 Mathematical Definition

A tree T is a connected acyclic graph where:

- There is exactly one root node with no parent
- Every other node has exactly one parent
- Nodes may have zero or more children

Formally: $T = (\text{root}, \text{children})$ where children is a set of trees.

7.2 Tree Terminology

Height: Maximum distance from root to any leaf. A tree with one node has height 0.

Depth of node: Distance from root to that node (root has depth 0).

Balanced tree: Height is $O(\log n)$ where n is the number of nodes.

Unbalanced tree: Height can be $O(n)$ (like a linked list).

Height h determines performance: a balanced tree with n nodes has height $\log_2(n)$.

7.3 Tree Traversals

Pre-order: Visit root, then left subtree, then right subtree. Used for copying trees.

In-order: Visit left subtree, then root, then right subtree. For BSTs, gives sorted output.

Post-order: Visit left subtree, then right subtree, then root. Used for deletion.

Level-order: Visit nodes level by level using a queue (BFS).

All traversals take $O(n)$ time since we visit each node exactly once.

Chapter 8: Binary Search Trees - Ordered Trees

8.1 Mathematical Definition

A Binary Search Tree (BST) is a binary tree where for each node N:

- All values in the left subtree are $< N.value$
- All values in the right subtree are $> N.value$

This property holds recursively for all subtrees.

8.2 BST Operations Complexity

Operation	Best Case	Worst Case	Average Case
Search	$O(\log n)$	$O(n)$	$O(\log n)$
Insert	$O(\log n)$	$O(n)$	$O(\log n)$
Delete	$O(\log n)$	$O(n)$	$O(\log n)$
Min/Max	$O(\log n)$	$O(n)$	$O(\log n)$

8.3 Why BST Search is Logarithmic

In a balanced BST, each comparison eliminates roughly half the remaining elements. If we have n elements, the number of times we can divide by 2 is $\log_2(n)$. For example, searching 1,000,000 elements takes at most $\log_2(1,000,000) \approx 20$ comparisons!

However: If the tree becomes unbalanced (like a linked list), $O(\log n)$ degrades to $O(n)$. This is why self-balancing trees (AVL, Red-Black) are important.

8.4 Example: BST Search Algorithm Principle

Mathematical Principle - Binary Search: Start at root. If target equals current node, found. If target $<$ current, search left subtree (eliminating right half). If target $>$ current, search right subtree (eliminating left half).

Each step eliminates approximately half the search space, giving us $\log_2(n)$ steps.

Chapter 9: Balanced Trees - AVL & Red-Black

9.1 The Balance Concept

Height balance: For each node, $|\text{height}(\text{left}) - \text{height}(\text{right})| \leq k$ (usually 1)

This constraint prevents degeneration into linked lists.

Key invariant: A balanced tree with n nodes has height $h = O(\log n)$.

9.2 AVL Trees

AVL Property: For every node, $|\text{height}(\text{left}) - \text{height}(\text{right})| \leq 1$

When insertion/deletion violates this, 'rotations' rebalance the tree.

AVL trees guarantee $O(\log n)$ for all operations.

Rotation Principle: A left rotation moves the right child up, right child becomes new parent. This changes the tree structure while maintaining BST order. After rotation, the height difference reduces.

9.3 Red-Black Trees

Red-Black Property: Each node is colored red or black with rules:

1. Root is black
2. Red nodes cannot have red children
3. All paths from node to leaves have equal number of black nodes

These rules ensure height $\leq 2 \cdot \log_2(n+1)$, guaranteeing $O(\log n)$ operations.

Red-Black trees require fewer rotations than AVL, making them faster in practice.

Chapter 10: Heaps - Priority-Based Structures

10.1 Mathematical Definition

A heap is a complete binary tree where each parent is $\geq$ all its children (max-heap) or $\leq$ all its children (min-heap).

Complete Binary Tree: All levels are fully filled except possibly the last level, which is filled left-to-right.

10.2 Array Representation

A heap can be stored in an array where for a node at index i :

- Parent is at index $\lfloor (i-1)/2 \rfloor$
- Left child is at index $2i+1$
- Right child is at index $2i+2$

This clever indexing scheme avoids pointer overhead and ensures cache locality.

10.3 Heap Operations

Operation	Time Complexity	Explanation
Insert	$O(\log n)$	Add to end, bubble-up along path to root
Extract-Max	$O(\log n)$	Remove root, move last element to root, bubble-down
Heapify (build)	$O(n)$	Convert array into heap in linear time
Peek-Max	$O(1)$	Root is maximum, no restructuring needed

10.4 Why Heapify is $O(n)$

Many mistakenly think heapify is $O(n \log n)$, but it's actually $O(n)$. Why?

The key insight: nodes near the bottom do little work (only a few comparisons), while nodes near the top do more work. Mathematically, the sum of work across all nodes is:

$$\text{Work} = \sum (\text{work at node } i) = \sum (\text{height of node } i) = O(n)$$

This is because about half the nodes are leaves (height 1), quarter are at height 2, etc.

10.5 Priority Queue Applications

Heaps implement priority queues where highest-priority element is served first. Applications include Dijkstra's algorithm (shortest paths), Huffman coding (compression), and event-driven simulation.

Chapter 11: Graphs - Networks & Relationships

11.1 Mathematical Definition

A graph $G = (V, E)$ consists of:

- V : set of vertices (nodes)
 - E : set of edges connecting vertices
- Edge $e = (u, v)$ connects vertices u and v .

Directed vs Undirected: In directed graphs, $(u \rightarrow v)$ is different from $(v \rightarrow u)$. In undirected graphs, edges are symmetric.

11.2 Graph Representations

Adjacency Matrix: $n \times n$ matrix where $M[i][j] = 1$ if edge exists from i to j . Space: $O(|V|^2)$, good for dense graphs, fast edge lookup.

Adjacency List: For each vertex, maintain list of adjacent vertices. Space: $O(|V| + |E|)$, good for sparse graphs.

11.3 Graph Traversal Algorithms

Breadth-First Search (BFS): Explore by levels using a queue. Time: $O(|V| + |E|)$, finds shortest paths in unweighted graphs.

Depth-First Search (DFS): Explore recursively using a stack. Time: $O(|V| + |E|)$, useful for detecting cycles and topological sorting.

11.4 Key Graph Concepts

Path: Sequence of vertices where consecutive vertices are connected by edges. Path length = number of edges.

Cycle: Path that starts and ends at same vertex with length ≥ 1 .

Connected: Undirected graph where path exists between every pair of vertices.

Strongly Connected: Directed graph where path exists from every vertex to every other vertex.

Chapter 12: Hash Tables - Constant-Time Lookup

12.1 The Hash Table Miracle

Hash tables achieve $O(1)$ average-case insertion, deletion, and search by using a hash function to convert keys into array indices. This is one of the most important inventions in computer science!

12.2 Mathematical Foundation

A hash function $h: \text{key_universe} \rightarrow \{0, 1, \dots, m-1\}$ maps any key to a slot in an array of size m .

Ideal property: h distributes keys uniformly across slots.

Collision: When $h(\text{key}_1) = h(\text{key}_2)$ for $\text{key}_1 \neq \text{key}_2$

12.3 Collision Resolution Techniques

Chaining: Each array slot points to a linked list of colliding elements. With m slots and n keys, average chain length = n/m . If we keep $n < m$, average search is $O(1)$.

Open Addressing: When collision occurs, probe for next empty slot (linear, quadratic, or double hashing). Load factor $\alpha = n/m$ must stay < 1 for efficiency.

12.4 Load Factor & Rehashing

As we insert elements, the load factor $\alpha = n/m$ increases. When α exceeds a threshold (typically 0.75), we double the table size and rehash all elements.

Mathematical guarantee: By maintaining $\alpha = O(1)$, insertion/deletion in hash tables stay $O(1)$ amortized.

Chapter 13: Sorting Algorithms - Arranging Data

13.1 The Sorting Problem

Input: An array A of n elements

Output: A permutation of A where $A[0] \leq A[1] \leq \dots \leq A[n-1]$

Lower Bound for Comparison-Based Sorting: Any comparison-based sorting algorithm must make at least $n \log n$ comparisons in the worst case. Why? Because there are $n!$ possible orderings, and each comparison gives at most 1 bit of information, requiring $\log_2(n!) \approx n \log n$ comparisons.

13.2 Classic Sorting Algorithms

Algorithm	Best Case	Average	Worst Case	Space	Stable?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	Yes

13.3 Divide and Conquer: Merge Sort Analysis

Principle: Divide array in half, sort each half, merge results.

Recurrence Relation: $T(n) = 2 \cdot T(n/2) + n$

- Dividing: $O(1)$
- Solving two subproblems: $2 \cdot T(n/2)$
- Merging: $O(n)$

Solution: By the Master Theorem, $T(n) = O(n \log n)$. The logic: we do n work at each of $\log n$ levels, totaling $n \cdot \log n$ work.

13.4 Quick Sort: Average Case Analysis

Principle: Choose pivot, partition array, recursively sort halves.

Best/Average Case: If pivot is near median, we get $T(n) = 2 \cdot T(n/2) + n = O(n \log n)$

Worst Case: If pivot is always smallest/largest, we get $T(n) = T(n-1) + n = O(n^2)$

In practice: Quick Sort is faster than Merge Sort due to cache locality and lower space usage, despite the same asymptotic complexity.

Chapter 14: Searching Algorithms - Finding Needles

14.1 Linear Search

Algorithm: Check each element sequentially until found or end reached.

Complexity: $O(n)$ best case (found immediately), $O(n)$ worst/average case

When to use: Small arrays or unsorted data where sorting cost isn't justified.

14.2 Binary Search

Precondition: Array must be sorted

Algorithm: Start at middle. If target found, done. If target < middle, search left half. If target > middle, search right half. Repeat.

Recurrence: $T(n) = T(n/2) + O(1)$

Complexity: $O(\log n)$

Why it works: Each comparison eliminates half the remaining elements. After k comparisons, we have $n/2^k$ elements left. When $n/2^k = 1$, we've found the answer. Solving: $2^k = n$, so $k = \log_2(n)$.

14.3 Search Trees for Range Queries

BSTs and balanced trees enable range queries: 'Find all elements between x and y in $O(\log n + k)$ time', where k is the number of results. This is faster than linear search when k is small.

Chapter 15: Recursion & Backtracking

15.1 Mathematical Foundation of Recursion

A recursive function is defined in terms of itself:

$f(n) = \text{base_case}$ (if $n = 0$ or 1) or $f(n) = \text{recursive_case}(f(n-1), f(n-2), \dots)$

Two requirements for correctness:

1. **Base case:** Problem with $n = 0$ or 1 can be solved directly
2. **Inductive step:** If we can solve for $n-1$, we can solve for n

15.2 Recurrence Relations

Recursive algorithms produce recurrence relations describing their time complexity.

Example: Fibonacci

$T(n) = T(n-1) + T(n-2) + O(1) \rightarrow \text{Exponential } O(\phi^n)$ where $\phi \approx 1.618$

Example: Binary Search

$T(n) = T(n/2) + O(1) \rightarrow O(\log n)$

Solution methods: Master Theorem, recursion tree method, or iteration method.

15.3 The Master Theorem

For recurrences $T(n) = a \cdot T(n/b) + f(n)$:

If $f(n) = O(n^{(\log_b(a) - \epsilon)})$: $T(n) = \Theta(n^{\log_b(a)})$

If $f(n) = \Theta(n^{\log_b(a)})$: $T(n) = \Theta(n^{\log_b(a)} \cdot \log n)$

If $f(n) = \Omega(n^{(\log_b(a) + \epsilon)})$: $T(n) = \Theta(f(n))$

The Master Theorem eliminates the need to solve recurrences algebraically for common patterns.

15.4 Backtracking Principle

Backtracking is a recursive strategy for exploring all possible solutions to a constraint satisfaction problem.

Principle: Build solution incrementally. At each step, check if current partial solution can lead to full solution. If not, 'backtrack' and try different choice. This prunes the search tree, avoiding exploring all $n!$ possibilities.

Time Complexity: Depends on problem, but exponential in worst case. However, pruning often reduces practical time significantly.

15.5 Example: N-Queens Problem

Problem: Place n queens on $n \times n$ chessboard so none attack each other.

Backtracking approach: Place queens row by row. For each row, try each column. Before placing, check if this violates constraints (attacking previous queens). If valid, continue to next row. If invalid or next row fails, backtrack and try next column.

Complexity: Without pruning: $O(n^n)$. With pruning: much better in practice.

Chapter 16: Dynamic Programming - Optimal Substructure

16.1 The Dynamic Programming Principle

Optimal Substructure: An optimal solution to a problem contains optimal solutions to subproblems.

Overlapping Subproblems: Same subproblems recur multiple times during computation.

Dynamic Programming exploits these properties by solving each subproblem once and storing the result, avoiding redundant computation.

16.2 Example: Fibonacci Numbers

Naive Recursion: $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$

Complexity: $O(\phi^n) \approx O(1.618^n)$ - exponential! Computing $\text{fib}(40)$ takes minutes.

Why exponential? $\text{fib}(n)$ is computed many times. For example, $\text{fib}(n-2)$ is computed both as part of $\text{fib}(n-1)$ and directly.

Dynamic Programming (Memoization): Compute $\text{fib}(1)$, $\text{fib}(2)$, ..., $\text{fib}(n)$ bottom-up, storing results.

Complexity: $O(n)$ - linear! Each number computed exactly once.

16.3 Two Approaches to DP

Top-Down (Memoization): Write recursive solution, but cache results. Natural formulation but may have overhead from recursion.

Bottom-Up (Tabulation): Compute results iteratively from base cases up to target. More efficient as it avoids recursion overhead and clearly shows computational order.

16.4 The Coin Change Problem

Problem: Given coins of denominations $d_1, d_2, \dots, d_k$ and target amount n , find minimum number of coins to make amount n .

Optimal Substructure: If we use coin d_i , remaining amount is $n - d_i$. Minimum coins for $n = 1 + \min(\text{minimum coins for each } n - d_i)$

DP Solution: $\text{dp}[i]$ = minimum coins for amount i

$\text{dp}[0] = 0$ (base case)

$\text{dp}[i] = \min(\text{dp}[i - d_j] + 1)$ for all valid coins d_j

Complexity: $O(n \times k)$ where k is number of coin types.

Chapter 17: Greedy Algorithms - Local Optimality

17.1 The Greedy Strategy

A greedy algorithm makes locally optimal choices at each step, hoping to find a global optimum.

When does Greedy work? Only for problems satisfying the 'greedy choice property' and optimal substructure. For arbitrary problems, greedy often fails (it's a heuristic, not guaranteed correct).

17.2 Example: Activity Selection Problem

Problem: Given n activities with start time s_i and end time e_i , select maximum number of non-overlapping activities.

Greedy Choice: Always pick activity ending earliest among remaining compatible activities.

Why it works: By picking the activity finishing earliest, we leave maximum room for subsequent activities. This greedy choice is optimal because it maximizes future options.

Complexity: $O(n \log n)$ if pre-sorted by end time, then $O(n)$ to select activities.

17.3 Huffman Coding: Optimal Compression

Problem: Assign variable-length binary codes to characters such that total encoding length is minimized.

Greedy Approach: Repeatedly combine two least-frequent characters into a tree node. This builds a binary tree where common characters are closer to root (shorter codes).

Why optimal: By combining least frequent characters first, we minimize their representation cost. Mathematical proof uses exchange argument: any other tree has equal or worse length.

17.4 Example: Fractional Knapsack

Problem: Given items with weight and value, fill knapsack of capacity W to maximize value. Items can be fractionally included.

Greedy Solution: Sort by value/weight ratio (value density), take items greedily in this order.

Optimality Proof: If optimal solution differs, swapping items by density order improves total value, contradicting optimality of the original solution.

Note: For 0/1 knapsack (can't take fractions), greedy fails; use dynamic programming instead.

Chapter 18: Advanced Graph Algorithms

18.1 Shortest Path: Dijkstra's Algorithm

Problem: Given weighted graph and source vertex s , find shortest path from s to all other vertices.

Greedy Strategy: Maintain set S of vertices whose shortest paths are known. Repeatedly pick vertex v not in S with minimum distance from s , add it to S . This works because we always process vertices in order of increasing distance.

Proof of Correctness: When we select vertex v , no shorter path exists to v through unprocessed vertices, because those vertices have greater distances.

Complexity: $O((|V|+|E|) \log |V|)$ with binary heap priority queue.

18.2 Minimum Spanning Tree: Kruskal's Algorithm

Problem: Given weighted undirected graph, find spanning tree with minimum total weight.

Greedy Strategy: Sort edges by weight. Greedily add edges if they don't create cycle (using union-find). Continue until we have $|V|-1$ edges.

Why Optimal: If we exclude any edge in the MST and add a heavier edge, total weight increases. So MST must include lightest edges possible without forming cycles.

Complexity: $O(|E| \log |E|)$ for sorting, $O(|E| \alpha(|V|))$ for union-find operations.

18.3 Maximum Flow: Ford-Fulkerson Method

Problem: Given flow network with source s and sink t , find maximum flow from s to t .

Key Idea: While there exists augmenting path from s to t in residual graph, push flow along that path (by minimum remaining capacity on that path).

Max-Flow Min-Cut Theorem: Maximum flow equals minimum cut capacity. This deep result connects two seemingly different problems.

Complexity: $O(|E| \times \text{max_flow})$ with DFS, $O(|V| \times |E|^2)$ with BFS (Edmonds-Karp).

Conclusion: Designing Efficient Algorithms

Key Takeaways

To design efficient algorithms:

1. Understand the Problem Mathematically

Express the problem formally. Identify the input size n and desired time/space complexity. Ask: 'What is the lower bound?' (what's theoretically necessary?)

2. Recognize Problem Patterns

Does it have optimal substructure? (Dynamic Programming)

Does it have greedy choice property? (Greedy)

Is it a shortest path variant? (Graph algorithms)

Can it be solved by dividing? (Divide and conquer)

3. Match Data Structures to Problems

Need fast lookup? Hash table ($O(1)$ average) or BST ($O(\log n)$ worst)

Need priority operations? Heap

Need ordering? Sorted array (binary search) or balanced tree

4. Analyze Complexity Rigorously

Write recurrence relations. Use Master Theorem. Count operations. Verify with examples.

5. Consider Trade-offs

$O(1)$ vs $O(\log n)$ vs $O(n)$: sometimes the constants matter more than asymptotic complexity

Time vs Space: can we speed up by using more memory?

Precomputation: can we compute once and answer queries fast?

The Algorithm Designer's Checklist

- ✓ Understand the problem and constraints
- ✓ Identify best data structure for your operations
- ✓ Sketch algorithm on paper (pseudocode, not code)
- ✓ Analyze worst-case and average-case complexity
- ✓ Prove correctness by induction or contradiction
- ✓ Verify with small examples
- ✓ Consider edge cases
- ✓ Compare with alternative approaches

Remember: The best algorithm is often a combination of techniques you've learned, adapted to your specific problem.

Final Wisdom

Mathematics in algorithms is not abstract theory—it's the language for precise thinking. Big O notation lets us ignore implementation details and focus on scalability. Recurrence relations let us understand recursive algorithms. Graph theory lets us model relationships. Proofs give us confidence that algorithms are correct.

By mastering these mathematical concepts without code, you gain insights that transcend programming languages. You can design efficient algorithms anywhere—on paper, in pseudocode, or in any programming language.